

TREKKING MANAGEMENT APPLICATION V2

PROJECT REPORT

STUDENT DETAILS

Name : Anish Bhattacharya
Roll Number : 23f2005522
Course Name : MAD2 Project
Email : 23f2005522@ds.study.iitm.ac.in

Brief Profile :

A learner pursuing the MAD2 course at IIT Madras, built full-stack web applications using Python (Flask) and Vue.js. This project is Version 2 of the Trekking Management Application originally developed during [MAD1](#) private repo .

PROJECT DETAILS

This project is a web-based Trekking Management Application designed to simplify the management of trekking activities for adventure organizations. Trekkers can register, browse and filter open treks, book slots, cancel bookings, view trekking history, and export history as CSV; Trek Staff can manage assigned treks, update slots and status, and manage participants; and Administrators can manage treks, staff, trekkers, bookings, statistics, and reports — each role interacting with the system based on assigned permissions.

The application follows a decoupled architecture: Flask REST API (backend) + Vue.js SPA (frontend), built using Python, Flask, SQLite, Redis, and Celery, with a Bootstrap 5 interface on the Vue frontend.

FRAMEWORKS AND LIBRARIES USED

- Backend: Python (Flask), Flask-SQLAlchemy (ORM), Flask-Migrate, Flask-JWT-Extended, Flask-CORS, Flask-SSE, Flask-Caching, Celery, Redis, Werkzeug
- Frontend: Vue 3, Vue Router, Pinia, Axios, Bootstrap 5, Bootstrap Icons, Vite
- Database: SQLite
- Background Jobs: Celery Worker, Celery Beat, Redis (broker + result backend)
- Other Tools: Flask Blueprints, JWT tokens, Server-Sent Events (SSE), MailHog (local email testing)

Why these?

Flask builds the lightweight REST API; Vue.js + Pinia handle the SPA frontend and centralized State Management. JWT secures auth, Celery + Redis run background jobs to reduce server load on time taking tasks , Flask-SSE pushes live messages when those tasks are done easily , and Redis caching improves API performance. SQLAlchemy, Migrate, Werkzeug, Bootstrap, and SQLite support data, schema, security, UI, and storage.

DEMO VIDEO

To view the demonstration video
[LINK](#)

ER DIAGRAM

To view the ER diagram
[LINK](#)

DATABASE SCHEMA

- User: id, username, email, phone, password, role (admin/trekker/staff), is_active, is_blacklisted, blacklisted_reason, created_at, updated_at
- Staff: id, user_id (FK → User), joining_date, experience, address, contact_number, staff_bio, Profile_status, created_at, updated_at
- Trek: id, name, location, difficulty, duration, total_slots, available_slots, price, description, image_url, starting_at, ending_at, status, assigned_staff_id (FK → Staff), created_at, updated_at
- Booking: id, user_id (FK → User), trek_id (FK → Trek), booking_date, status, payment_status, amount_paid, booking_cancel_date, booking_cancel_reason, created_at, updated_at

- Notification (V2): id, user_id (FK → User), message_text, is_read, type_of_notification, status, created_at, updated_at

Purpose: Backend audit log only. Records are created by Celery tasks (daily reminders, monthly reports, CSV export). Not exposed via any read API and not displayed in a notification panel on the frontend.

NOTE : User-facing alerts are delivered through Flask-SSE as temporary flash messages.

- ExportJob (V2): id, user_id (FK → User), file_path, status, type_of_report, created_at, updated_at

Purpose: Tracks asynchronous CSV export requests for trekkers. Stores job status (pending/processing/completed/failed) and the saved file path once export finishes.

Trekkers can view export history and download completed files from the History page.

- **ReportLogs** (V2): id, type_of_report, report_summary, status, created_at, updated_at [standalone — no FK]

Purpose: Internal developer/ops log for scheduled report job execution (e.g. monthly admin report). Used for backend traceability only — not linked to users and not surfaced in any admin or user interface.

DATABASE RELATIONSHIPS

user (1) ————— (1) staff === One user with role staff has one staff profile.
FK: staffs.user_id → users.id

staff (1) ————— (many) trek === One staff member can be assigned to many treks.
FK: treks.assigned_staff_id → staffs.id

user (1) ————— (many) booking === One trekker can have many bookings.
FK: bookings.user_id → users.id

trek (1) ————— (many) booking === One trek can have many bookings.
FK: bookings.trek_id → treks.id

user (1) ————— (many) notification === Backend audit trail per user (write-only log).
FK: notifications.user_id → users.id
Note: No read API; live user alerts use Flask-SSE flash messages, not this table.

user (1) ————— (many) export_job === One trekker can trigger multiple CSV exports.
FK: export_jobs.user_id → users.id
Note: Actively used — trekker History page lists jobs and downloads completed CSVs.

SYSTEM ARCHITECTURE

____The application follows a decoupled architecture: Flask REST API (backend) + Vue.js SPA (frontend)____

BACKEND (Flask REST API)

main.py	– App entry point, blueprint registration, cache init, DB seed
model/model.py	– SQLAlchemy models, enums, relationships, constraints
routes/	– REST API endpoints via Flask Blueprints (auth, admin, staff, trekker, health)
utils/	– JWT role_required decorator, email utils, cache utility
tasks.py	– Celery background tasks (reminders, reports, CSV export)
celery_worker.py	– Celery app configuration and Beat schedule
extensions/cache.py	– Flask-Caching Redis setup
templates/	– HTML email templates (upcoming treks, monthly report)
db/seed_data.py	– Programmatic admin and dummy data seeding
instance/Trek.db	– SQLite database file
migrations/	– Flask-Migrate (Alembic) database migrations

FRONTEND (Vue.js SPA)

main.js	– Vue app entry point and plugin setup
App.vue	– Root application component
router/index.js	– Vue Router with role-based route guards
views/	– Role-specific pages (admin, staff, trekker, auth, home)
components/	– Reusable UI (modals, navbar, booking modal, loaders)
stores/	– Pinia state management (admin, staff, trekker, notifications)
utils/axioUtil.js	– Axios instance with JWT interceptor
utils/logout.js	– Logout and SSE disconnect handler

FEATURES IMPLEMENTED

- JWT-based authentication with role-based access control (Admin, Trek Staff, Trekker)
 - Trekker self-registration; Staff created by Admin with approval workflow; Admin predefined login
 - Role-specific Vue.js dashboards with route guards and post-login redirects
 - Admin: dashboard stats, trek CRUD, staff assignment, search, blacklist/deactivate, bookings, reports
 - Staff: assigned treks only, dashboard counts, status updates (open/ongoing/closed/completed), slot management, participant management
 - Trekker: browse/filter open treks, book/cancel treks, profile update, booking history, trekking history
 - Duplicate booking prevention, overbooking prevention, trek/booking status tracking
 - Celery scheduled jobs: daily trek reminder emails + monthly admin report (HTML email)
 - Async CSV export of trekking history — ExportJob tracks status and file path; SSE flash message alerts trekker when download is ready (no persistent notification UI)
 - NotificationModel and ReportLogsModel maintained as backend audit logs for job traceability
 - Redis API caching on trek listings with TTL (120s) and cache invalidation on data changes
 - Responsive UI with Bootstrap 5 across all role-specific Vue pages
-

AI/LLM tools (including GitHub Copilot and ChatGPT) were used for approximately 15–20% of the development process.

The assistance was mainly for:

- *GitHub Copilot* : was used during this project, mostly for syntax correction, refactoring, and milestone mapping — that is, checking the requirements document to see if everything was completed according to the requirement document and if I had missed anything.

For Example — There was also a case where my own implementation did not match the requirement and then I refactored the code myself to remove staff self-registration and keep registration open only for Trekkers.

For Example — I had missed error handling and try-except in a few routes. The copilot highlighted the inconsistency, and I corrected those parts on my own.

For example — Before implementing each route controller, I planned the feature flow, validated the design, and Once the workflow was clear, I built it independently using the planned architecture, mainly for booking rules (open trek only, slot limits, duplicate prevention), staff/trekker approval/blacklist/reject flows, and the trek status lifecycle.

- *ChatGPT (and layout reference from MAD1)* : was used to draft Vue components with hardcoded dummy data for layout reference with bootstrap classes . After that, I reworked those pages on my own, using Vue3 (v-if, v-for, @click, etc.) and kept the UI consistent across admin, trekker, and staff sections.
- *MAD 2 class Docs* [[LINK](#), [LINK](#), [LINK](#) , [LINK](#)] : was used as a reference to understand how to set up Redis and Celery, how Celery Beat works with MailHog, how to send live notifications with SSE for the trekker export job, and how to implement Redis caching. — The actual code implementation was written by me.

Apart from these, all core logic and features were implemented by me.

END OF REPORT

Student: Anish Bhattacharya | Roll No: 23f2005522 | MAD2 Project
